The Journal of Open Source Software

CMBLensing.py: A Python Package for MAP Estimation of CMB Fields and Cosmological Parameter Sampling

Zane Blood [1*], **Kimmy Wu** [1*], **and Fei Ge** [1*]

1 California Institute of Technology ***** These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

CMBLensing.py (Blood, 2026) is the Python migration of CMBLensing.jl (Millea, 2017), a Julia package for simulating Cosmic Microwave Background (CMB) fields, gravitational lensing, Maximum A Posteriori (MAP) estimates of CMB fields, and cosmological parameter sampling. CMBLensing.py is written in JAX (Bradbury et al., 2018), with most of its methods being Just-In-Time (JIT) compilable and executable on both CPU and GPU. The Python port reproduces the full functionality of the Julia version and has been extensively tested for regressions. We have also extended the Julia sampler - which originally sampled the tensor-to-scalar ratio r and the lensing potential band power A_ϕ (Millea et al., 2020) - to jointly sample any subset of 5 of the Lambda Cold Dark Matter (LCDM) cosmological parameters, taking a prior value on the optical depth $\tau_{\rm reio}$.

Statement of Need

We migrated CMBLensing.jl into Python for better integration with other Python-based machine learning and inference projects, planned and ongoing, that our group is developing. Calling CMBLensing.jl from Python previously required the `juliacall` (JuliaPy, 2020) package and forced the user to write Julia inside a Python string, making debugging and iteration slow and tedious. We expect having the full functionality present natively in Python will be of use to many CMB researchers working in the general area of cosmological inference.

The LCDM parameter sampler is also a novel addition, expanding on the algorithm of (Millea et al., 2020) for jointly sampling r and A_ϕ. As far as we know, it is the first Gibbs sampler to jointly infer LCDM parameters, the unlensed CMB, and the lensing potential directly at the field level from CMB maps with CMB lensing included in the forward model. We have used it to jointly infer the subset $\left[\ln(A_s * 10^{10}), 100 * \theta_{MC}, \Omega_c h^2\right]$ from lensed temperature maps at varying white noise levels, with the other LCDM parameters fixed at fiducial values.

State of the Field

The `map_joint` method computes MAP estimates of the unlensed CMB fields and the lensing potential from input noisy lensed data maps. Here we compare its run time and accuracy between the two language implementations. Both packages contain other CMB analysis tools, but `map_joint` is a good all around benchmark since it brings many methods together into a single function.

We ran `map_joint` on temperature plus polarization data and stored the cached run time per language, averaged over 10 input data maps and 10 trials per map, at several square map sizes.

DRAFT

We list the experimental details in Table 1 and the results in Table 2. The two implementations are just as fast (within 1 standard deviation) at smaller map sizes, but at the largest map size tested Python is up to 4 times slower than Julia.

Table 1: Configuration of the `map_joint` run time and accuracy comparison experiment.

Quantity	Value
Data	Temperature plus polarization (`IP`)
Map sizes	128^2, 256^2, 512^2, 1024^2 pixels
Resolution	2.5 arcminutes per pixel
White noise level	$3\ \mu K$ arcminutes in T, $\sqrt{2} \times 3\ \mu K$ arcminutes in E and B
$1/f$ noise	ℓ-knee $= 100$, α-knee $= 3$
Beam	None (FWHM $= 0$)
Mask	Fourier space low-pass held at unity through $\ell = 2950$, then a cosine ramp down to zero at $\ell = 3000$
`map_joint num_steps`	30
Data maps per map size	10
Trials per data map	10

Table 2: Cached `map_joint` run times (mean $\pm$ standard deviation) versus map size for temperature plus polarization (`IP`) data.

Map Size	Julia Average (s)	Python Average (s)	Python / Julia
128^2	99.5 ± 41.7	192.9 ± 82.5	1.9
256^2	550.4 ± 342.0	769.4 ± 300.0	1.4
512^2	$2,332.0 \pm 537.0$	$4,799.0 \pm 1,440.0$	2.1
1024^2	$6,712.0 \pm 1,450.0$	$25,700.0 \pm 3,930.0$	3.8

Figure 1 below benchmarks the accuracy of the two language implementations. We compare the average fractional difference between the Julia MAP estimates and the ground truth fields against the same quantity for Python. Both language implementations agree to within numerical precision.

DRAFT

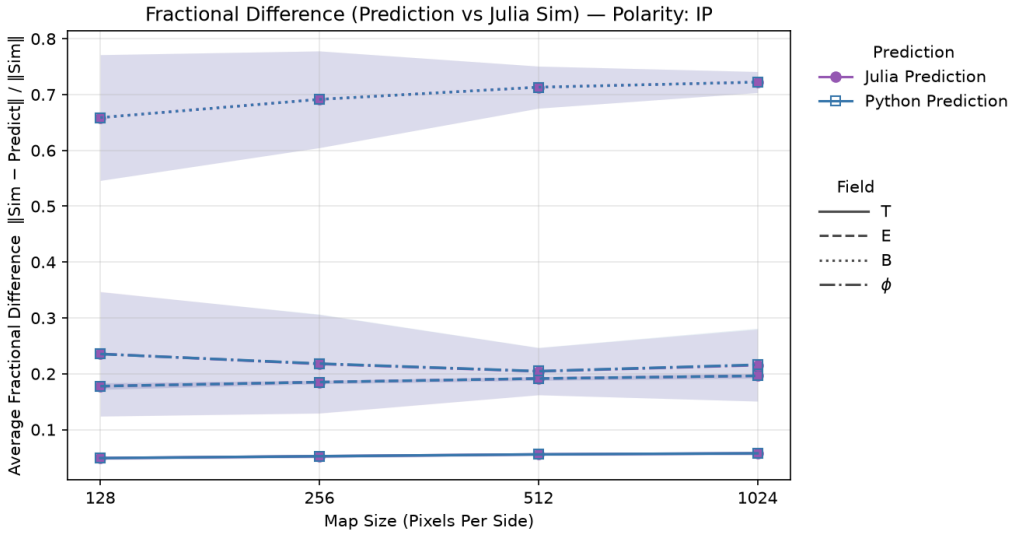


Figure 1: Fractional difference between the MAP estimates from either language and the simulation ground truth. Both use ground truth fields from the Julia simulation code, eliminating the possibility that the Python simulation and MAP code are wrong in a self-consistent way.

Figure 2 below visually compares the Python output MAP estimates of the temperature and polarization fields as well as the lensing potential relative to the ground truth for a single representative trial.

DRAFT

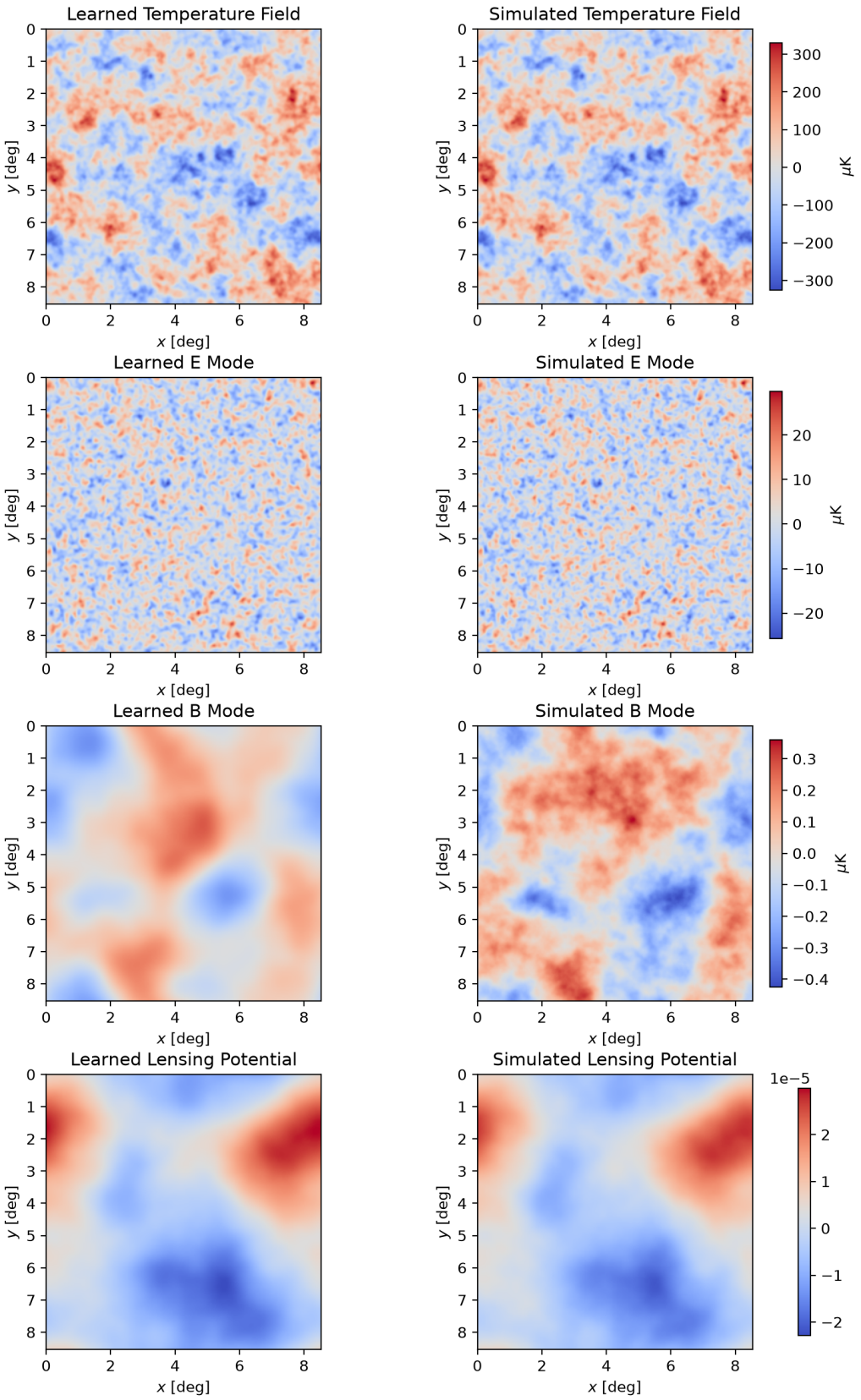


Figure 2: Visual comparison of MAP estimates and ground truth fields for a single representative 256 x 256 pixel temperature map with resolution of 2 arcminutes per pixel and white noise level of 3 μK-arcmin.

Software Design

CMBLensing.py was migrated method by method, ensuring the two implementations produced outputs identical to within numerical precision for the same inputs. One of the first methods ported was `lense_flow`, which iteratively lenses a CMB field given the unlensed fields and lensing potential (Millea et al., 2019). The Wiener filter, likelihood, and gradient methods followed, packaged into the main `map_joint` entry point.

Before implementing the new LCDM sampler, the machinery which jointly sampled (r, A_ϕ) was ported and confirmed to learn the same distributions between implementations for the same input maps. The step sampling those parameters was then swapped for one drawing LCDM parameter samples, leaving the neighboring field and lensing potential samplers untouched.

Unit tests in the `\tests` folder validate that corresponding methods in the two implementations produce identical outputs.

Research Impact Statement

Besides making CMBLensing natively available in JAX, the sampling algorithm in `sample_lcdm.py` represents (to the best of our knowledge) the first field-level joint inference of the LCDM parameters, unlensed CMB, and lensing potential that includes lensing in the forward model. Other work, such as (Crespi et al., 2025), has inferred these parameters from map level data but does not include lensing in the forward model or sampling of the lensing potential. The Gibbs sampler we employ in this work follows the logic in the pseudo-code block below:

Initialize the lensing potential ϕ and the LCDM parameters $\vec{\theta}$
for $k = 1, \ldots, N_{\text{steps}}$ **do**
    Sample $f \sim P(f \mid d, \phi, \vec{\theta})$ via conjugate gradient
    Sample $\phi \sim P(\phi \mid d, f, \vec{\theta})$ via Hamiltonian Monte Carlo
    for each $\theta_i \in \vec{\theta}$ **do**
        Sample $\theta_i \sim P(\theta_i \mid d, f, \phi, \{\theta_j\}_{j \neq i})$ via 1-D Metropolis-Hastings
    end for
end for

We find that learning $[\ln(A_s * 10^{10}), 100 * \theta_{MC}, \Omega_c h^2]$ jointly is relatively easy for smaller temperature maps. The sampler could in theory be applied to the full set of 6 LCDM parameters, but in practice the map sizes needed to break the degeneracies and produce meaningful distributions with small widths become prohibitively large.

Tables 3 and 4 below outline the configuration for 3 experiments we did at 2.5 , 5, and 10 μK-arcmin white noise levels to jointly infer $[\ln(A_s * 10^{10}), 100 * \theta_{MC}, \Omega_c h^2]$ on 128 x 128 square pixel temperature maps. Table 3 lists the settings shared by all three experiments and Table 4 the sampler settings tuned per noise level.

Table 3: Configuration shared by the joint $[\ln(A_s * 10^{10}), 100 * \theta_{MC}, \Omega_c h^2]$ sampling experiments run with `sample_joint` at three different white noise levels.

Quantity	Value
Data	Temperature only (`I`)
Map size	128^2 pixels
Resolution	2.5 arcminutes per pixel
White noise level	2.5, 5, and 10 μK-arcminutes
$1/f$ noise	None
Beam	None (FWHM $= 0$)

DRAFT

Quantity	Value
Mask	None
Sampled parameters	$\ln(A_s * 10^{10})$, $100 * \theta_{\mathrm{MC}}$, $\Omega_c h^2$
Parameters held at ground truth	$\Omega_b h^2$, n_s, τ_{reio}
Ground truth $\ln(A_s * 10^{10})$	3.218387
Ground truth $100 * \theta_{MC}$	1.031732
Ground truth $\Omega_c h^2$	0.109381
Ground truth $\Omega_b h^2$	0.022386
Ground truth n_s	0.959814
Ground truth τ_{reio}	0.05
Search range, θ_{MC}	$[0.9328, 1.1452]$
Search range, $\log(A_s)$	$[2.661635, 3.782861]$
Search range, $\Omega_c h^2$	$[0.085, 0.155541]$
LCDM starting points	Drawn at random inside the search ranges
θ step	Sequential 1-D random walk Metropolis Hastings
ϕ step	HMC with `num_steps` $= 10$
ϕ initialization	Zero
Iterations with θ held fixed	100
Data map realizations	10
Sub-chains per data map	5
Iterations per sub-chain	6000
Thinning	By each sub-chain's integrated autocorrelation time
Convergence	Average Gelman-Rubin $\hat{R} < 1.05$

Table 4: Sampler settings tuned per white noise level for the experiments of Table 3.

Quantity	2.5 μK-arcmin	5 μK-arcmin	10 μK-arcmin
$\ln(A_s * 10^{10})$ proposal width	2×10^{-2}	2×10^{-2}	3.5×10^{-2}
$100 * \theta_{\mathrm{MC}}$ proposal width	2×10^{-3}	3×10^{-3}	4×10^{-3}
$\Omega_c h^2$ proposal width	1.5×10^{-3}	8×10^{-4}	3.5×10^{-3}
ϕ HMC `step_size`	0.0175	0.05	0.05
Burn-in per sub-chain (samples)	1000	600	600

After running the sampling algorithm for the configuration in Tables 3 and 4 for each noise level, burn-in was removed, the sub-chains thinned by their integrated auto-correlation time and concatenated into 10 processed chains per map per noise level. A Gaussian kernel density estimate on each chain gave 10 "per-map" distributions per noise level. The product of these per-map distributions is taken and re-normalized. Its mean represents our best estimate, across all input data maps, of a given parameter's fiducial value at the specified noise level of the experiment. Table 5 below shows the results for each learned parameter across each noise level.

Table 5: Experimental results of the joint $[\ln(A_s * 10^{10}), 100 * \theta_{MC}, \Omega_c h^2]$ sampling experiments across noise level. Mode, Mean, and Standard Deviation are the values for the corresponding learned product distribution. Z-Score is the difference of ground truth and product distribution mean normalized by the width of the product distribution.

Parameter	Statistic	2.5 μK-arcmin	5 μK-arcmin	10 μK-arcmin
$\ln(A_s * 10^{10})$	Ground truth	3.218387	3.218387	3.218387
	Mode	3.222778	3.225215	3.227765
	Mean	3.222439	3.223605	3.225800

DRAFT

Parameter	Statistic	$2.5\ \mu K$-arcmin	$5\ \mu K$-arcmin	$10\ \mu K$-arcmin
	Standard Deviation	0.005379	0.006163	0.007177
	Z-Score	-0.753277	-0.846637	-1.032873
$100 * \theta_{MC}$	Ground truth	1.031732	1.031732	1.031732
	Mode	1.031954	1.031182	1.030323
	Mean	1.031783	1.031375	1.030534
	Standard Deviation	0.000667	0.001037	0.001412
	Z-Score	-0.076728	0.344775	0.848557
$\Omega_c h^2$	Ground truth	0.109381	0.109381	0.109381
	Mode	0.109226	0.108974	0.108345
	Mean	0.109262	0.108894	0.108174
	Standard Deviation	0.000671	0.000870	0.001470
	Z-Score	0.177027	0.560639	0.821229

For the given noise level, map size, resolution, and sampler inputs we can compute a Fisher forecast using APIs available in CMBLensing.py for the expected lower bound on an individual map's learned uncertainty and compare it to the empirical widths. Figure 4 shows a triangle plot with Fisher in red and experimental results in blue. The experimental widths are only slightly larger, indicating our sampler performs close to optimally.

DRAFT

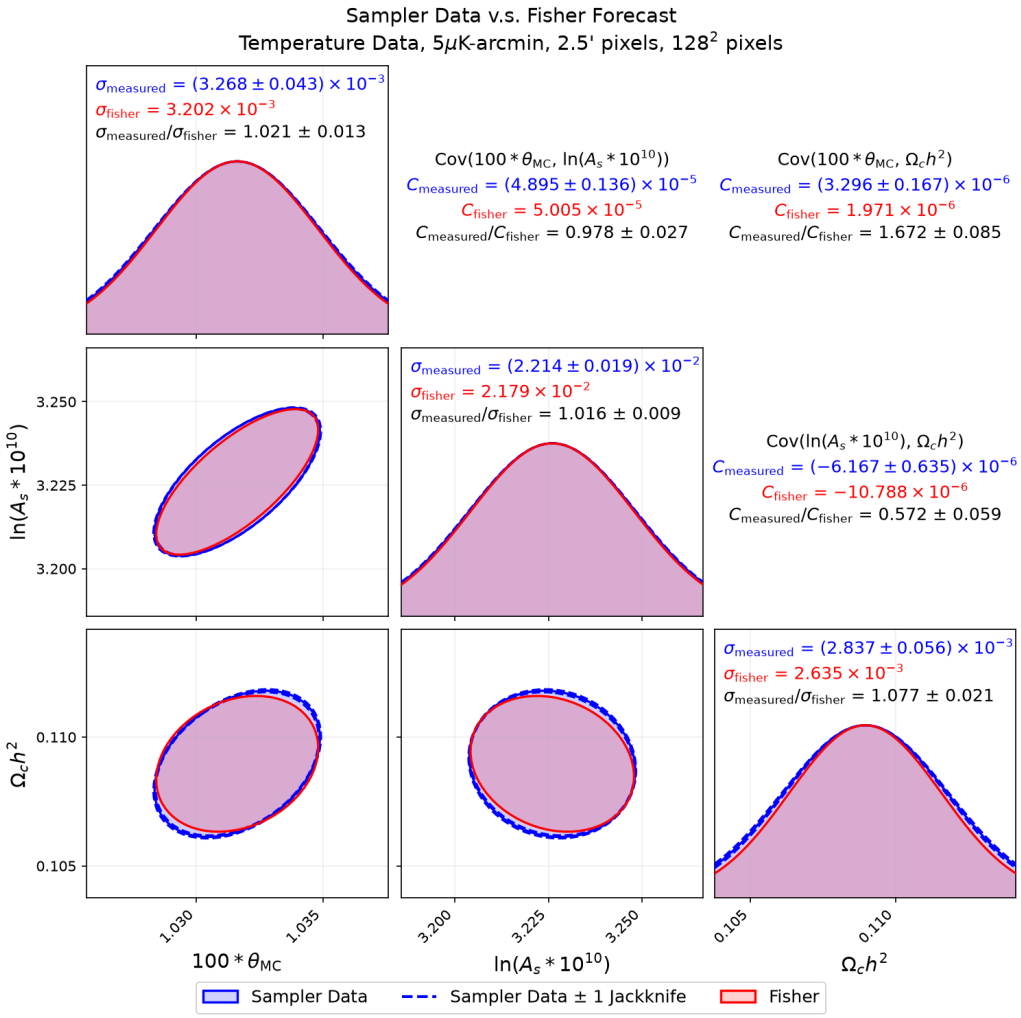


Figure 3: Triangle plot comparing the empirically found distributions and correlations with the Fisher forecast.

Limitations

In theory our sampler could jointly infer all 6 LCDM parameters at once, but in practice the largest subset we could learn in testing was just 3. Many of the LCDM parameters are strongly degenerate, and breaking those degeneracies requires increasing the map size. One iteration takes less than 3 seconds for 128 x 128 temperature-only maps with 4 `cpus-per-task` on Caltech's Resnick HPC (Caltech Resnick High Performance Computing Center, 2026) or Texas A&M's ACES cluster (Boerner et al., 2023), but running on larger maps can significantly slow down the sampler. The major bottle neck for larger maps is the time complexity of the HMC step for sampling the lensing potential. Other works such as (Crespi et al., 2025) have found success in using more advanced sampling schemes such as Micro Canonical Langevin Monte Carlo. Exploring different samplers for the lensing potential may be an avenue for future time complexity improvements.

Perhaps the most troublesome limitation is the current mixing scheme we employ to decouple the field and lensing potential in our sampler. We use the mixing matrices G and D defined in (Millea et al., 2020) which are computed in terms of the noise covariance matrix. For these mixing matrices, high noise levels yield good mixing, but also large widths in the computed

distributions for the most difficult parameters to learn such as n_s or $\Omega_b h^2$. Learning these parameters would require a lower overall noise level, but dropping the noise diminishes the mixing matrices so that the effective sample size tends toward zero. We empirically find that the mixing matrices perform optimally at the 5 μK noise level and their performance decreases quickly below that threshold. A mathematically justified mixing scheme decoupled from the experiment's noise would be a major area for future improvement.

AI Usage Disclosure

Claude Code was used for code generation in parts of the code base and for preparation of this manuscript. All AI generated code was reviewed by a human before being merged.

Acknowledgements

The computations presented here were conducted in part at the Resnick High Performance Computing Center, a facility supported by the Resnick Sustainability Institute at the California Institute of Technology.

This work also used the ACES cluster at Texas A&M University through allocation PHY260360 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program (Boerner et al., 2023), which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

References

Blood, Z. (2026). CMBLensing.py. In *GitHub repository*. GitHub. https://github.com/zaneblood1/CMBLensing.py

Boerner, T. J., Deems, S., Furlani, T. R., Knuth, S. L., & Towns, J. (2023, July). ACCESS: Advancing innovation: NSF's advanced cyberinfrastructure coordination ecosystem: Services & support. *Practice and Experience in Advanced Research Computing (PEARC '23)*. https://doi.org/10.1145/3569951.3597559

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Caltech Resnick High Performance Computing Center. (2026). *Caltech resnick high performance computing center*. https://www.hpc.caltech.edu

Crespi, A., Bonici, M., Loureiro, A., Ruiz-Zapatero, J., Sladoljev, I., Li, Z., Bayer, A., Millea, M., & Seljak, U. (2025). *Flinch: A differentiable framework for field-level inference of cosmological parameters from curved sky data*. https://arxiv.org/abs/2510.26691

JuliaPy. (2020). PythonCall.jl. In *GitHub repository*. GitHub. https://github.com/JuliaPy/PythonCall.jl

Millea, M. (2017). CMBLensing.jl. In *GitHub repository*. GitHub. https://github.com/marius311/CMBLensing.jl

Millea, M., Anderes, E., & Wandelt, B. D. (2019). Bayesian delensing of CMB temperature and polarization. *Physical Review D*, *100*(2). https://doi.org/10.1103/physrevd.100.023509

Millea, M., Anderes, E., & Wandelt, B. D. (2020). Sampling-based inference of the primordial CMB and gravitational lensing. *Physical Review D*, *102*(12). https://doi.org/10.1103/physrevd.102.123542

DRAFT